

PROJECT AND TEAM INFORMATION

Project Title

Project Proposal: Urban Emergency Pre-emption Engine (UEPE)

Student / Team Information

<i>Team Name:</i> <i>DSCPP-III-2026-T319</i>	<i>Avengers</i>
Team member 1 (Team Lead)	<i>Pranav Pandey</i> <i>252027984@geu.ac.in</i> <i>2027984</i> 
Team member 2	<i>Mohit Singh Basera</i> <i>@geu.ac.in</i> <i>2027920</i> 
Team member 3	<i>Riddhi</i> <i>2512520008@geu.ac.in</i> <i>2028472</i> 
	

PROPOSAL DESCRIPTION

Motivation

Anyone who's been stuck behind an ambulance at a red light — or worse, in one — knows how much time gets lost at intersections that just don't know an emergency is happening. Systems exist that fix this (RF emitters, GPS-linked signal overrides) but they're expensive, closed-source, and basically invisible in how they make decisions. We wanted to build something that shows the actual logic: given a dispatch, how do you compute the fastest route, and how do you get every signal along that route to clear in time — especially when two emergency vehicles are converging on the same intersection at once?

State of the Art / Current solution

The commercial answer here is Opticom-style systems — emitters on the vehicle talk to a receiver at the intersection and force a green. Works well, but it's hardware-heavy and you can't really see what's happening inside. Adaptive systems like SCATS adjust for regular traffic congestion but don't treat an ambulance any differently from a delivery truck. We're not trying to replace any of this — we're building a simulator that shows how a graph of the road network, a priority queue for competing requests, and a hash table for fast intersection lookups can reproduce the decision-making part of these systems, cleanly enough that you can trace exactly why a given signal turned green when it did.

Project Goals and Milestones

General Goals:

- Design and implement an emergency-vehicle signal pre-emption simulator using custom-built data structures in C/C++.
- Demonstrate how data structure choice (graphs for road networks, priority queues for request scheduling, hash tables for intersection lookup, min-heaps for shortest-path/ETA computation) affects pre-emption latency and correctness.
- Apply C++ OOP principles to structure the system into reusable classes (Intersection, EmergencyVehicle, RouteRequest, SignalController, PreemptionManager).

Milestones:

1. Milestone 1 – Core Network Model: Define classes/structs for Intersection, RoadSegment, and EmergencyVehicle; represent the city road network as a graph (adjacency list) and implement basic vehicle entry/exit event logging.
2. Milestone 2 – Routing Engine: Implement Dijkstra's/A* shortest-path algorithm (using a min-heap/priority queue) to compute the fastest route and ETA for an emergency vehicle from source to destination.
3. Milestone 3 – Pre-emption Request Queue: Implement a priority queue to arbitrate between multiple simultaneous emergency requests at the same intersection (priority by vehicle type, proximity, wait time), plus a hash table mapping intersection IDs to live signal state for O(1) lookup.
4. Milestone 4 – Signal State & Conflict Resolution: Implement a SignalController class per intersection that pre-empts normal cycles, handles conflicting simultaneous requests (e.g., two ambulances crossing the same junction), and restores normal cycling after clearance; log all overrides.
5. Milestone 5 – Reporting & Final Integration: Generate reports (average pre-emption delay saved, most-congested intersections, busiest corridors) using sorting algorithms; combine modules into a single CLI (or optional simple dashboard); write documentation and prepare a demo.

Project Approach

We plan to build UEPE using C for the core data-structure implementation (graph, heap, hash table — for low-level control and performance) combined with C++ for the OOP interface layer.

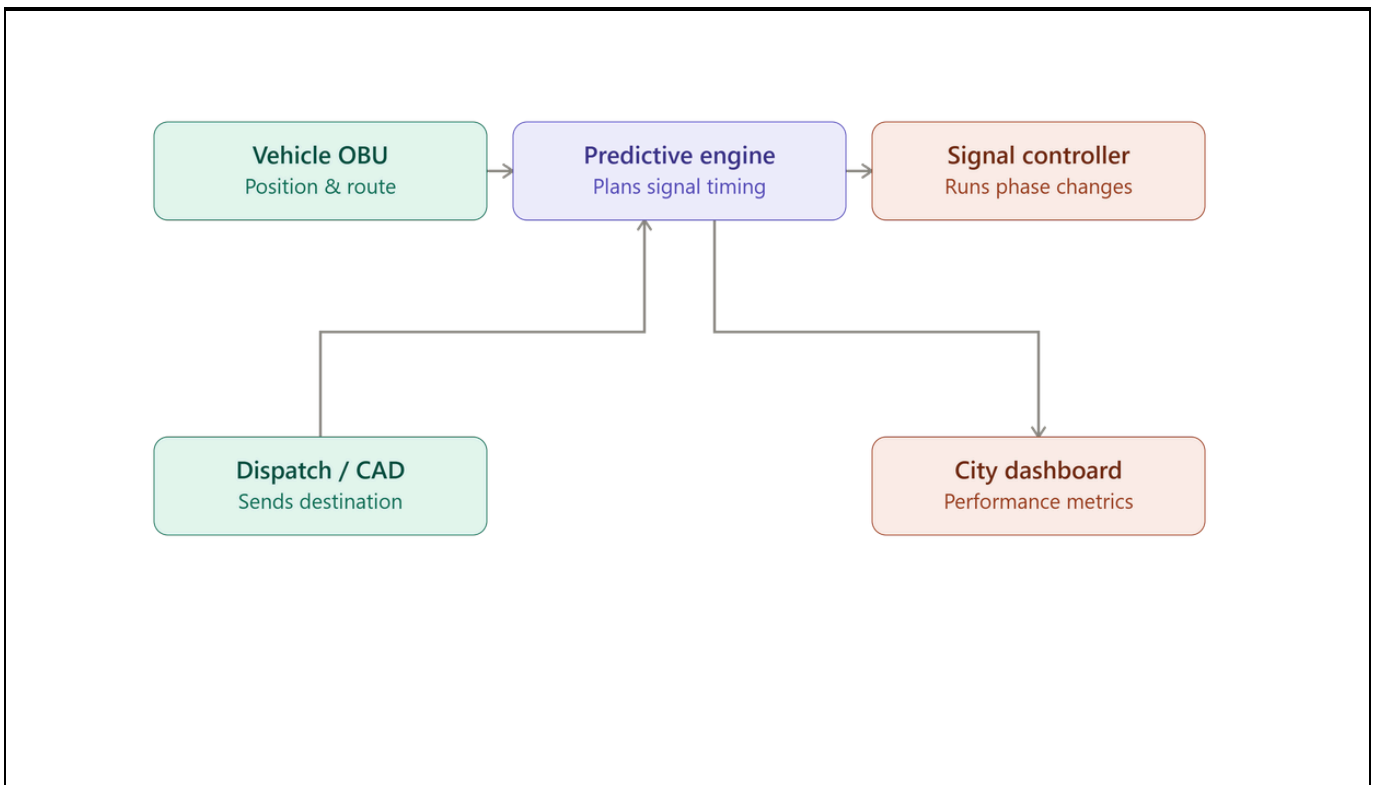
Key components:

- **Road Network Graph:** Adjacency-list graph representing intersections (nodes) and road segments (weighted edges by distance/travel time).
- **Routing Module:** Dijkstra's algorithm using a custom binary min-heap for shortest-path/ETA computation per emergency vehicle.
- **Pre-emption Priority Queue:** A priority queue ordering competing pre-emption requests by vehicle priority class and estimated arrival time.
- **Intersection Registry:** A hash table (custom implementation) mapping intersection IDs to SignalController objects for $O(1)$ average lookup, insert, and update.
- **Event Queue:** A FIFO queue processing incoming vehicle-dispatch and intersection-clearance events in temporal order.
- **Sorting/Reporting:** Sorting algorithms (quicksort/mergesort) to generate reports like "corridors with highest pre-emption frequency" or "average time saved per dispatch."

Platform & Technologies:

- **Language:** C (data structures) + C++ (OOP interface layer)
- **Development environment:** VS Code / Code::Blocks with GCC/G++ compiler
- **Version control:** Git/GitHub
- **Optional:** Simple command-line menu; if time permits, a basic dashboard (Qt or Flask/C++ backend) to visualize the road network and live pre-emption state
- **Testing:** Simulated city-grid datasets (CSV of intersections, road segments, and dispatch events) to validate routing/pre-emption logic — no real traffic-authority data or live hardware required

System Architecture (High Level Diagram)



Project Outcomes:

The final deliverable is a fully functional, console-based UEPE application implemented in C and C++ that models a city road network as a graph, computes optimal emergency routes, arbitrates competing pre-emption requests via a priority queue, overrides and restores signal states through a hash-indexed intersection registry, and generates activity reports (average delay saved, busiest corridors, pre-emption frequency by intersection). The system will demonstrate practical use of graphs, heaps/priority queues, hash tables, and queues, along with a clean C++ OOP class hierarchy (Intersection, EmergencyVehicle, RouteRequest, SignalController, PreemptionManager). Deliverables include the complete source code (hosted on GitHub), a sample simulated city-grid dataset used for testing, a project report documenting design decisions and data structure trade-offs, and a live demo showing core functionality (routing, pre-emption arbitration, and reporting).

Assumptions

We assume a single-city, simulated-grid environment (no live GPS/RF hardware integration), and that vehicle dispatch and intersection data are supplied through simulated CSV/log input for development and testing rather than live traffic-controller hooks, keeping the project portable and safe to demo. We assume network sizes are moderate (up to a few hundred intersections, a few thousand dispatch events), so in-memory data structures are sufficient. The project is scoped strictly to simulation, routing, and pre-emption decision logic — it does not involve controlling actual traffic hardware or municipal systems.

References

- *GeeksforGeeks — data structure implementation guides*
- *Deitel & Deitel, C++ How to Program*
- *Global Traffic Technologies Opticom — emergency vehicle pre-emption architecture reference*
- *Cormen, Leiserson, Rivest, Stein — Introduction to Algorithms*